Lecture # 03
Concurrent systems: Types of processors (RISC & CISC)

PARALLEL AND DISTRIBUTED COMPUTING

CS-482

What is Concurrency?

Concurrency in computing refers to the capability of an OS to handle more than one task or process at the same time, thereby enhancing efficiency and responsiveness. It may be supported by multi-threading or multi-processing whereby more than one process or threads are executed simultaneously or in an interleaved fashion.

Thus, more than one program may run simultaneously on shared resources of the system, such as CPU, memory, and so on. This helps optimize performance and reduce idle times while improving the responsiveness of applications, generally in multitasking contexts. Good concurrency handling is crucial for deadlock situations, race conditions, and usually also for uninterrupted execution of tasks. It helps in techniques like coordinating the execution of processes, memory allocation, and execution scheduling for maximizing throughput.

What is Concurrency in OS?

Concurrency in an operating system refers to the ability to execute multiple processes or threads simultaneously, improving resource utilization and system efficiency. It allows several tasks to be in progress at the same time, either by running on separate processors or through context switching on a single processor. Concurrency is essential in modern OS design to handle multitasking, increase system responsiveness, and optimize performance for users and applications.

Motivation for Concurrent Systems

There are several motivations for allowing concurrent execution:

- **Physical resource Sharing:** Multiuser environment since hardware resources are limited
- **Logical resource Sharing:** Shared file (same piece of information)
- **Computation Speedup:** Parallel execution
- **Modularity:** Divide system functions into separation processes

Relationship Between Processes of Operating System

The Processes executing in the operating system is one of the following two types:

- Independent Processes
- Cooperating Processes

Independent Processes

Its state is not shared with any other process.

- The result of execution depends only on the input state.
- The result of the execution will always be the same for the same input.
- The termination of the independent process will not terminate any other.

Cooperating Processes

Its state is shared along other processes.

- The result of the execution depends on relative execution sequence and cannot be predicted in advanced(Non-deterministic).
- The result of the execution will not always be the same for the same input.
- The termination of the cooperating process may affect other process.

Process Operation in Operating System

Most systems support at least two types of operations that can be invoked on a process creation and process deletion.

Process Creation

A parent process and then children of that process can be created. When more than one process is created several possible implementations exist.

- Parent and child can execute concurrently.
- The Parents waits until all of its children have terminated.
- The parent and children share all resources in common.
- The children share only a subset of their parent’s resources.

Process Termination

A child process can be terminated in the following ways:

A parent may terminate the execution of one of its children for a following reasons:

1. The child has exceeded its allocation resource usage.
2. The task assigned to its child is no longer required.

If a parent has terminated than its children must be terminated.

Principles of Concurrency

Both interleaved and overlapped processes can be viewed as examples of concurrent processes, they both present the same problems.

The relative speed of execution cannot be predicted. It depends on the following:

- The activities of other processes
- The way operating system handles interrupts
- The scheduling policies of the operating system

Problems in Concurrency

- **Sharing global resources:** Sharing of global resources safely is difficult. If two processes both make use of a global variable and both perform read and write on that variable, then the order in which various read and write are executed is critical.
- **Optimal allocation of resources:** It is difficult for the operating system to manage the allocation of resources optimally.
- **Locating programming errors:** It is very difficult to locate a programming error because reports are usually not reproducible.
- **Locking the channel:** It may be inefficient for the operating system to simply lock the channel and prevents its use by other processes.

Advantages of Concurrency

- **Running of multiple applications:** It enable to run multiple applications at the same time.
- **Better resource utilization:** It enables that the resources that are unused by one application can be used for other applications.
- **Better average response time:** Without concurrency, each application has to be run to completion before the next one can be run.
- **Better performance:** It enables the better performance by the operating system. When one application uses only the processor and another application uses only the disk drive then the time to run both applications concurrently to completion will be shorter than the time to run each application consecutively.

Drawbacks of Concurrency

- It is required to protect multiple applications from one another.
- It is required to coordinate multiple applications through additional mechanisms.
- Additional performance overheads and complexities in operating systems are required for switching among applications.
- Sometimes running too many applications concurrently leads to severely degraded performance.

Issues of Concurrency

- **Non-atomic:** Operations that are non-atomic but interruptible by multiple processes can cause problems.
- **Race conditions:** A race condition occurs of the outcome depends on which of several processes gets to a point first.
- **Blocking:** Processes can block waiting for resources. A process could be blocked for long period of time waiting for input from a terminal. If the process is required to periodically update some data, this would be very undesirable.
- **Starvation:** Starvation occurs when a process does not obtain service to progress.
- **Deadlock:** Deadlock occurs when two processes are blocked and hence neither can proceed to execute.

Conclusion

Concurrency in Operating Systems refers to the major quark forming a basis of design that allows more than one process or thread to execute with others. This improves system efficiency since the CPU is utilized to its maximum capacity, hence enhancing response time and supporting multitasking. It also gives way to a lot of complexity regarding race conditions, deadlocks, and resource contention. This concurrency will be effectively utilized through the utilization of management techniques, such as process synchronization, mutual exclusion, and deadlock avoidance strategies that will ensure stability and predictability of behavior within the system.

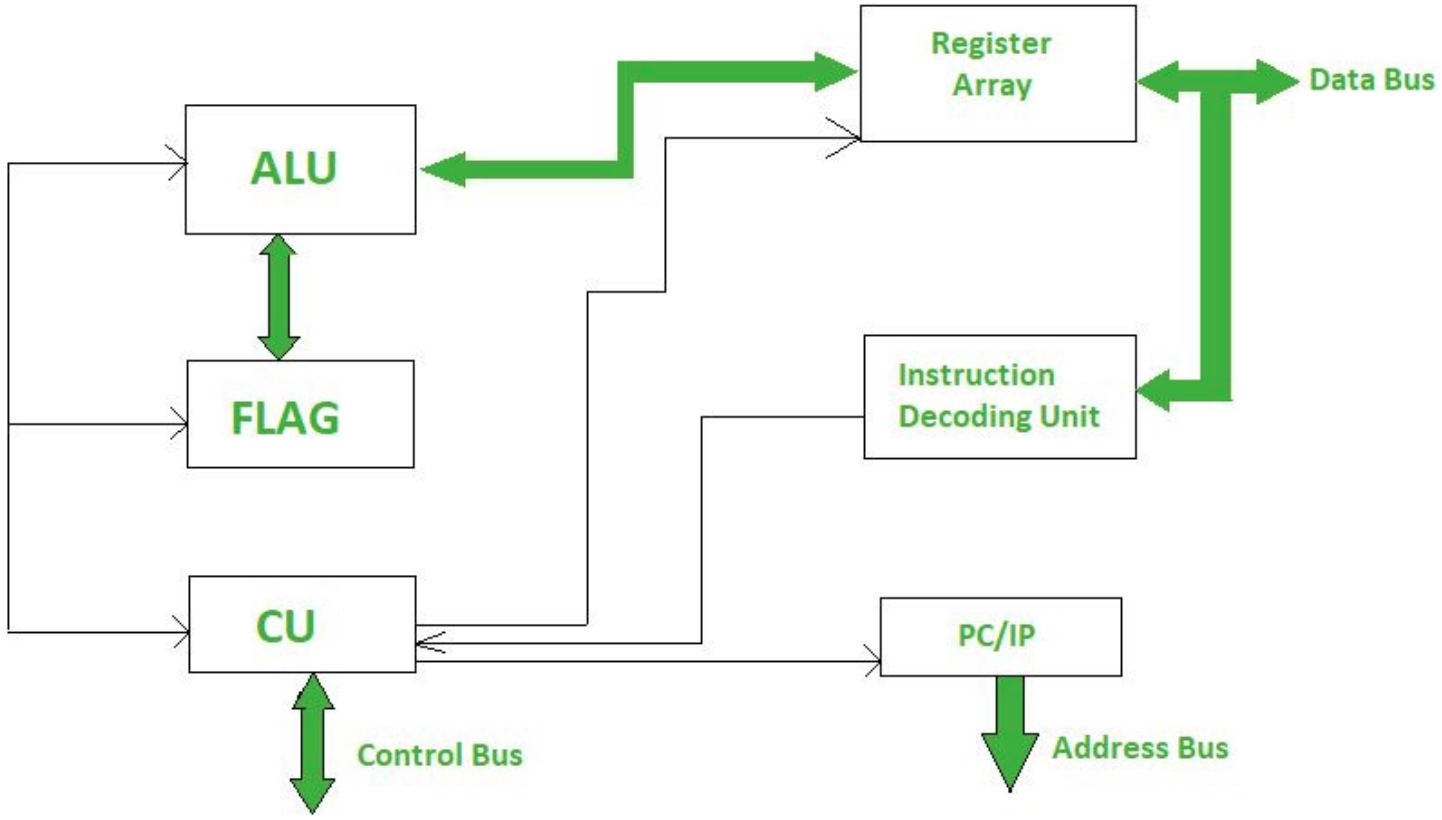
How is Concurrency different from Parallelism?

Concurrency will address a number of tasks all at once but does not necessarily execute them all simultaneously, while parallelism refers to the execution of more than one task at the same time, utilizing different processors or cores

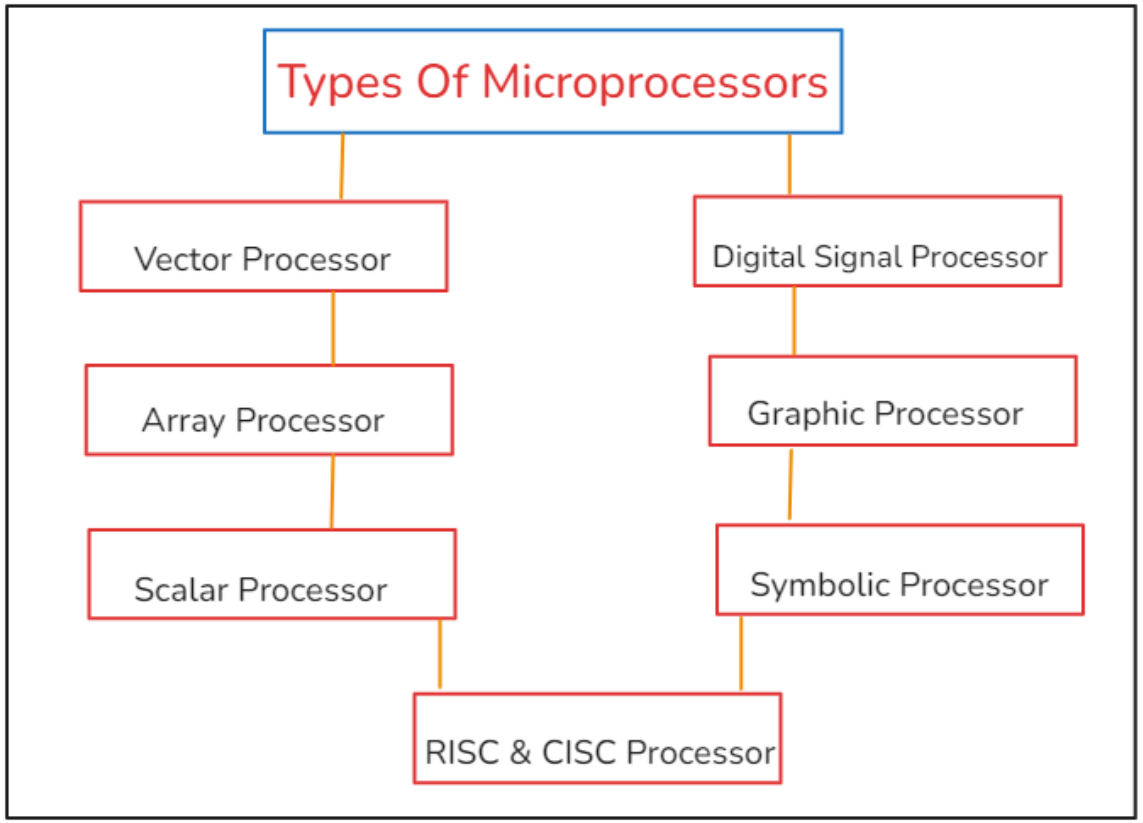
What is a Microprocessor?

A microprocessor is a computer processor that is found in most modern personal computers, smartphones, and other electronic devices. It is a central processing unit (CPU) that performs most of the processing tasks in a computer system. The microprocessor is a key component of a computer, as it controls the fetching, decoding, and execution of instructions that are stored in memory. You can say that microprocessor is used as the brain of the computing devices which control overall execution and operations. The development of microprocessors has played a significant role in the evolution of computers and has made it possible for them to become smaller, faster, and more powerful over time.

What is a Microprocessor?



Types of Microprocessors



Vector Processor

A vector processor is a type of central processing unit (CPU) that is designed to perform mathematical operations on arrays of data, called vectors, more efficiently than a scalar processor, which operates on single data elements. Vector processors can perform operations on multiple data elements simultaneously, which can lead to faster and more efficient processing. Vector processors can be found in some supercomputers and servers, as well as in some specialized graphics processing units (GPUs).

Array Processor or SIMD Processor

Array processors are also designed for vector computations. The difference between an array processor and a scalar processor is that a vector processor uses multiple vector pipelines whereas an array processor employs a number of processing elements to operate in parallel. Array processors can also be found in some supercomputers and servers, as well as in some specialized graphics processing units (GPUs). Array processors are different from scalar processors, which operate on single data elements and are more common in general-purpose computing applications.

Scalar Processor

It is a processor which executes scalar data. The simplest scalar processor makes the processing of only integer instruction using fixed point operands. A powerful scalar processor makes the processing of both integers as well as floating point numbers. It contains an integer ALU and a floating-point unit (FPU) on the same CPU chip. A scalar processor may be a CISC or RISC processor. A super scalar processor contains multiple pipelines and executes more than one instruction per clock cycle.

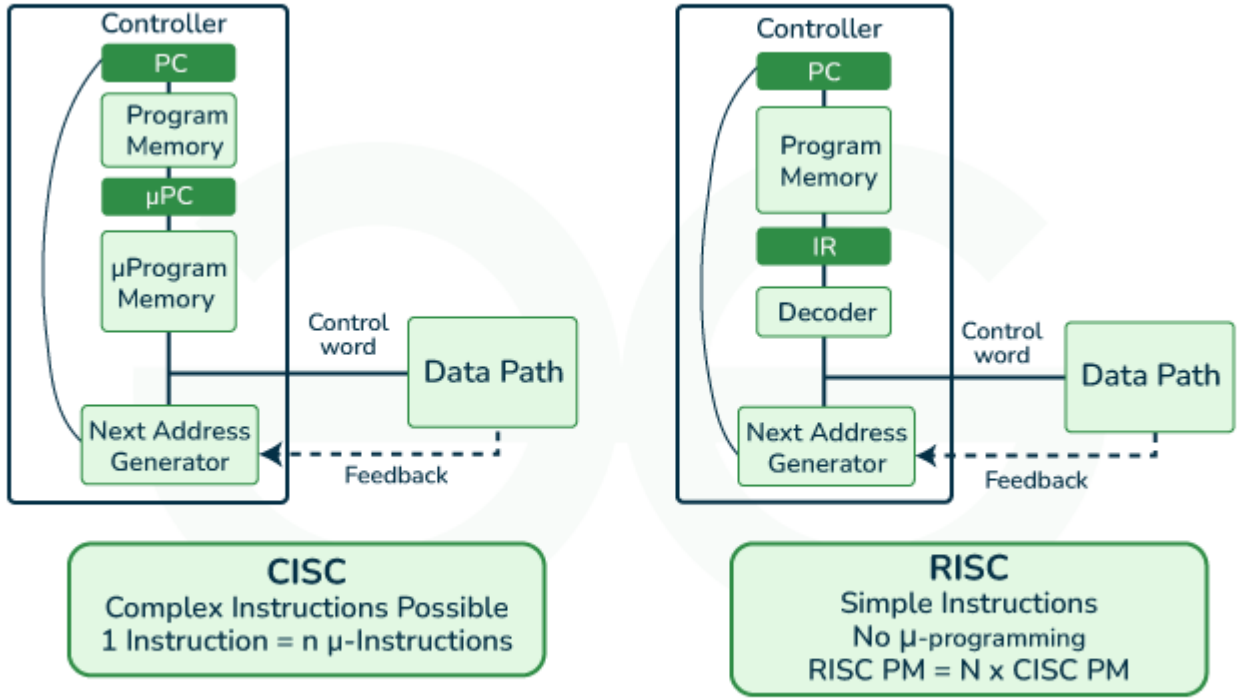
RISC and CISC Processor

RISC (Reduced Instruction Set Computing) and CISC (Complex Instruction Set Computing) are two approaches to designing a central processing unit (CPU), which is the main component of a computer that performs most of the processing tasks. RISC processors have a smaller, simpler instruction set, which means they have fewer types of instructions that they can execute. This makes them easier to design and manufacture, and allows them to execute instructions faster than CISC processors. RISC processors are typically used in devices that require high performance and low power consumption, such as smartphones and tablets. RISC processors are faster and more efficient than CISC processors.

CISC processors, on the other hand, have a larger and more complex instruction set, which means they can execute a wider range of instructions. This makes them more versatile, but also more expensive and slower to execute instructions than RISC processors. CISC processors are typically used in devices that require more flexibility, such as desktop computers and servers.

RISC and CISC in Computer Organization

RISC is the way to make hardware simpler whereas CISC is the single instruction that handles multiple work.



Reduced Instruction Set Architecture (RISC)

The main idea behind this is to simplify hardware by using an instruction set composed of a few basic steps for loading, evaluating, and storing operations just like a load command will load data, a store command will store the data.

Characteristics of RISC

- Simpler instruction, hence simple instruction decoding.
- Instruction comes undersize of one word.
- Instruction takes a single clock cycle to get executed.
- More general-purpose registers.
- Simple Addressing Modes.
- Fewer Data types.
- A pipeline can be achieved.

Advantages of RISC

- **Simpler instructions:** RISC processors use a smaller set of simple instructions, which makes them easier to decode and execute quickly. This results in faster processing times.
- **Faster execution:** Because RISC processors have a simpler instruction set, they can execute instructions faster than CISC processors.
- **Lower power consumption:** RISC processors consume less power than CISC processors, making them ideal for portable devices.

Disadvantages of RISC

- **More instructions required:** RISC processors require more instructions to perform complex tasks than CISC processors.
- **Increased memory usage:** RISC processors require more memory to store the additional instructions needed to perform complex tasks.
- **Higher cost:** Developing and manufacturing RISC processors can be more expensive than CISC processors.

Complex Instruction Set Architecture (CISC)

The main idea is that a single instruction will do all loading, evaluating, and storing operations just like a multiplication command will do stuff like loading data, evaluating, and storing it, hence it’s complex.

Characteristics of CISC

- Complex instruction, hence complex instruction decoding.
- Instructions are larger than one-word size.
- Instruction may take more than a single clock cycle to get executed.
- Less number of general-purpose registers as operations get performed in memory itself.
- Complex Addressing Modes.
- More Data types.

Advantages of CISC

- **Reduced code size:** CISC processors use complex instructions that can perform multiple operations, reducing the amount of code needed to perform a task.
- **More memory efficient:** Because CISC instructions are more complex, they require fewer instructions to perform complex tasks, which can result in more memory-efficient code.
- **Widely used:** CISC processors have been in use for a longer time than RISC processors, so they have a larger user base and more available software.

Disadvantages of CISC

- **Slower execution:** CISC processors take longer to execute instructions because they have more complex instructions and need more time to decode them.
- **More complex design:** CISC processors have more complex instruction sets, which makes them more difficult to design and manufacture.
- **Higher power consumption:** CISC processors consume more power than RISC processors because of their more complex instruction sets.

CPU Performance

Both approaches try to increase the CPU performance

- **RISC:** Reduce the cycles per instruction at the cost of the number of instructions per program.

$$CPU\ Time = \frac{Seconds}{Program} = \frac{Instructions}{Program} X \frac{Cycles}{Instructions} X \frac{Seconds}{Cycle}$$

- **CISC:** The CISC approach attempts to minimize the number of instructions per program but at the cost of an increase in the number of cycles per instruction.

Example

Earlier when programming was done using assembly language, a need was felt to make instruction do more tasks because programming in assembly was tedious and error-prone due to which CISC architecture evolved but with the uprise of high-level language dependency on assembly reduced RISC architecture prevailed.

Suppose we have to add two 8-bit numbers:

- **CISC approach:** There will be a single command or instruction for this like ADD which will perform the task.

- **RISC approach:** Here programmer will write the first load command to load data in registers then it will use a suitable operator and then it will store the result in the desired location.

So, add operation is divided into parts i.e. load, operate, store due to which RISC programs are longer and require more memory to get stored but require fewer transistors due to less complex command.

RISC vs CISC

RISC	CISC
Focus on software	Focus on hardware
Uses only Hardwired control unit	Uses both hardwired and microprogrammed control unit
Transistors are used for more registers	Transistors are used for storing complex Instructions
Fixed sized instructions	Variable sized instructions
Can perform only Register to Register Arithmetic operations	Can perform REG to REG or REG to MEM or MEM to MEM
Requires more number of registers	Requires less number of registers
Code size is large	Code size is small
An instruction executed in a single clock cycle	Instruction takes more than one clock cycle
An instruction fit in one word.	Instructions are larger than the size of one word
Simple and limited addressing modes.	Complex and more addressing modes.

RISC vs CISC

RISC	CISC
RISC is Reduced Instruction Cycle.	CISC is Complex Instruction Cycle.
The number of instructions are less as compared to CISC.	The number of instructions are more as compared to RISC.
It consumes the low power.	It consumes more/high power.
RISC is highly pipelined.	CISC is less pipelined.
RISC required more RAM .	CISC required less RAM.
Here, Addressing modes are less.	Here, Addressing modes are more.